

PRACTICA 1 SISTEMAS INFORMÁTICOS

GRUPO: 1322

NUMERO DE PAREJA: 8

AUTORES: JORGE PALOMINO Y RAÚL MATELLANO

INTRODUCCIÓN Y DISEÑO

El objetivo principal ha sido diseñar un sistema de almacenamiento de textos compartidos y desarrollar la API

En lugar de programar una aplicación todo en uno, lo hemos implementado basándonos en microservicios, lo que nos aporta modularidad y escalabilidad del sistema

Para que los servicios puedan comunicarse con el exterior, hemos desarrollado una API usando Python y Quart junto con el formato JSON, donde la API actúa como interfaz de comunicación

IMPLEMENTACION DE MICROSERVICIOS

Hemos dividido el código en dos servicios independientes

User.py es el servicio que gestiona los registros, inicios de sesión y cambios de contraseña del usuario desde el puerto 5050

File.py por su parte es un servicio que se encarga de crear, leer y borrar, al igual que cambiar la visibilidad de los documentos

En ambos se ha utilizado Quart. Hemos importado Quart y jsonify para levantar el servidor y devolver las respuestas HTTP con la estructura JSON. Además, validamos las rutas desde la terminal con comandos curl

Para lo relacionado con la autenticación y la seguridad, queríamos mantener esa independencia entre los dos servicios. Por eso, en vez de obligar al servicio de ficheros a consultar al de usuarios cada vez que llega una petición, configuramos una variable de entorno compartida. Con esta variable y el id del usuario generamos los tokens para validar la identidad de forma local y sin sobrecargar la red

PERSISTENCIA DE DATOS

Para guardar la información hemos utilizado el sistema de archivos local creando una carpeta llamada almacenamiento. Donde dentro de esa carpeta se crearían dos directorios más, usuarios y ficheros, para guardar usuarios y documentos respectivamente

Los usuarios se guardan creando un archivo .json para cada usuario registrado, mientras que los ficheros se meten dentro de un subdirectorio nombrado con el UID de cada persona

Para saber si un texto lo puede ver cualquiera o es privado para el dueño, no se abre el txt, sino que se genera un pequeño archivo paralelo con extensión .meta que contiene un JSON indicando si el archivo es público o privado

CONTENEDORES Y DESPLIEGUE

Para el despliegue del sistema hemos decidido usar contenedores Docker ya que nos permite empaquetar todo el código y las dependencias en un formato estándar

Para preparar las imágenes de nuestros servicios hemos escrito un archivo Dockerfile. Este archivo contiene las instrucciones para crear una imagen en Docker. En nuestro caso establecemos la imagen base de Python y marcamos el directorio de trabajo "WORKDIR/app", y utilizaremos el comando "RUN pip install" para que instale todo lo que se necesita, que está escrito en el archivo requirements.txt

Para gestionar la ejecución hemos utilizado Docker Compose, que nos proporciona un fichero de configuración para arrancar múltiples contenedores a la vez. Ambos microservicios se construyen a partir del mismo Dockerfile, pero los independizamos utilizando la instrucción Command para que uno ejecute Python user.py y el otro Python file.py. Para poner todo el sistema en marcha de golpe, basta con escribir el comando docker-compose up --build

PRUEBAS AUTOMATICAS

Por último, hemos creado el script cliente.py para realizar las pruebas, hace uso de la librería "requests" de Python para utilizar la API simulando que fuese un usuario real, haciendo uso de 19 pruebas